



NestJS Course

Day [2]



Agenda

- **DTO**
- **Pipes**
- **DB connection (TypeORM)**
- **Nest CLI**
- **Guards**
- **Interceptors**



DTO



DTO

What are DTOs?

Data Transfer Objects (DTOs) define how data is sent/received by your API.

Why Use DTOs?

Validation: Ensure data is correct (e.g., valid email, strong password).

Clarity: Explicitly declare what your API expects/returns.

Documentation: Auto-generate API docs (Swagger) from DTOs.

Note : Install these packages

```
npm i --save class-validator class-transformer
```



Separation Between API and Database

Your database entity and API contract often evolve independently.
DTOs allow you to expose only what the client needs—while keeping
your database structure **private**.



Database Entity (Internal)

Used by the database / ORM

```
@Entity()
export class User {
  id: number;           // Primary key
  email: string;        // User email
  password: string;     // Hashed password
  role: string;         // e.g. 'admin'
  isActive: boolean;    // Account status
  createdAt: Date;      // Creation timestamp
  updatedAt: Date;      // Last update
}
```



Contains sensitive and internal fields
not meant for clients

Expose only
what the client
needs



Keeps your data
model private
and secure

API DTO (External)

Used in requests / responses



```
export class CreateUserDto {
  email: string;
  password: string;
}
```

```
export class UserResponseDto {
  id: number;
  email: string;
  createdAt: Date;
}
```



Only exposes safe and relevant fields
to the outside world



Key Takeaway

DTOs act as a boundary between your API and database.
They give you the freedom to change your database structure
without breaking your API contract.

- ✓ Hide internal fields
- ✓ Protect sensitive data
- ✓ Evolve independently
- ✓ Deliver a clean API



Pipes



Pipes

A pipe is a class annotated with the `@Injectable()` decorator, which implements the `PipeTransform` interface.

Pipes have two typical use cases:

- **transformation**: transform input data to the desired form.
- **validation**: evaluate input data and if valid, simply pass it through unchanged; otherwise, throw an exception

Nest comes with built-in pipes, **ValidationPipe**, **ParseIntPipe** and more; see this [link](#)



Connect to Database



Install typeorm and postgres

```
npm i --save @nestjs/typeorm typeorm pg
```



Nest CLI



Guards



Guards

A guard is a class annotated with the `@Injectable()` decorator, which implements the `CanActivate` interface.

Guards have a single responsibility. They determine whether a given request will be handled by the route handler or not, depending on certain conditions. Ex: `Permission`, `Role`, `ACL`, etc.

This is often referred to as authorization, has typically been handled by `middleware` in traditional Express applications.

But middleware, by its nature, is dumb. It doesn't know which handler will be executed after calling the `next()` function. On the other hand, Guards have access to the `ExecutionContext` instance and thus know exactly what's going to be executed next.



Interceptors



Interceptors

An Interceptor is a class annotated with the `@Injectable()` decorator, which implements the `NestInterceptor` interface.

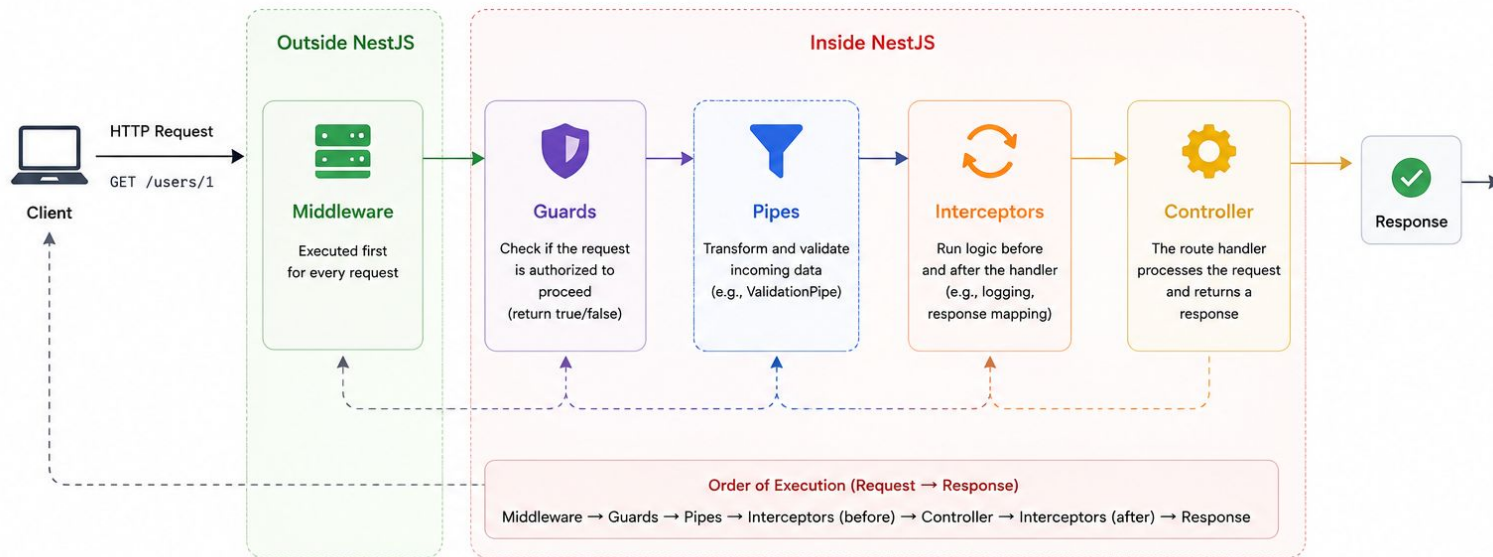
What can interceptors do ?

- bind extra logic before / after method execution
- transform the result returned from a function
- transform the exception thrown from a function
- extend the basic function behavior
- override a function execution on specific conditions (e.g., for **caching** purposes)



NestJS Request Lifecycle

Where Middleware, Guards, Interceptors and Pipes are executed



Middleware

- Executes before the request reaches NestJS
- Good for logging, parsing, CORS, etc.
- Applies to all routes

Guards

- Runs after middleware
- Checks authorization / authentication
- Can block the request

Pipes

- Runs before the controller handler
- Validates and transforms incoming data
- E.g., `ValidationPipe`, `ParseIntPipe`

Interceptors

- Runs around the handler
- Can modify request before it reaches the handler
- Can modify response before it's sent

Controller

- Handles the request
- Business logic lives here
- Returns data or response



Note: Exception Filters run if an exception is thrown at any point in the pipeline.



Lab [2]

- Use DTOs in create/update todo.
- Add validation on your inputs.
- Connect your application with Postgres DB.
- Update the CRUD operations to handle the operations in the database.



Thank You !!